

Wykrywanie anomalii w obrazach

Krok 3: Nienadzorowane wykrywanie za pomocą autoenkodera

Piotr Zapła

Bezpieczeństwo uczenia maszynowego

Sprawozdanie opracowano przy wsparciu modelu Claude Opus 4.7

maj 2026

Streszczenie

Celem eksperymentu było sprawdzenie, czy autoenkoder uczony wyłącznie na danych czystych wykrywa nieznany atak lepiej niż klasyfikator nadzorowany zbudowany w Kroku 1. Oba modele przebadano na zbiorze MNIST oraz na ukrytym zbiorze Pythia. Wyniki dla MNIST jednoznacznie potwierdziły hipotezę: na ataku znanym klasyfikatorowi obie metody są niemal równe, natomiast na ataku nieznanym autoenkoder osiąga miarę F1 na poziomie 0,975, podczas gdy klasyfikator spada do 0,244. Dla zbioru Pythia obie metody działają na poziomie losowym, co wskazuje, że różnica między klasami nie jest zakodowana w statystyce pikseli.

Spis treści

1	Wprowadzenie	1
2	Cel eksperymentu i hipoteza	1
3	Metodyka	2
3.1	Zbiory danych	2
3.2	Role partycji ataków	2
3.3	Modele	3
3.4	Wykrywanie anomalii przez błąd rekonstrukcji	3
3.5	Przebieg eksperymentu	3
3.6	Metryki oceny	4
4	Wyniki	4
4.1	MNIST — porównanie podstawowe	4
4.2	MNIST — analiza rozszerzona	4
4.3	Pythia — wyniki	7
5	Dyskusja	10
5.1	Atak znany — cena podejścia nienadzorowanego	10
5.2	Atak nieznany — potwierdzenie hipotezy	10
5.3	Co wykrywa każda z metod	10
5.4	Pythia — dlaczego obie metody zawodzą	11
6	Wnioski	11
	Załącznik — pliki wynikowe	11

1 Wprowadzenie

Wykrywanie anomalii w danych obrazowych to jedno z podstawowych zadań w bezpieczeństwie uczenia maszynowego. Chodzi o to, aby automatycznie odróżnić dane *normalne* od danych zmodyfikowanych — zaszumionych, celowo spreparowanych lub pochodzących z innego źródła niż dane, na których system ma działać.

W poprzednich krokach projektu zbudowano i przebadano *klasyfikator nadzorowany*. Krok 1 ujawnił jego główne ograniczenie — tak zwane *założenie zamkniętego świata*: model uczy się rozpoznawać konkretny, znany atak i zawodzi, gdy pojawia się atak nowego typu, do którego nie ma się jak odwołać. Krok 2 sprawdził, czy pomaga trenowanie na wielu typach ataków naraz — okazało się, że pomaga jedynie częściowo, a dla niektórych ataków w ogóle.

Krok 3 podchodzi do problemu odwrotnie. Zamiast uczyć model *jak wygląda atak*, uczymy go *jak wygląda norma* — a wszystko, co od normy odbiega, traktujemy jako anomalię. Narzędziem jest **autoenkoder**: sieć, która uczy się kompresować i odtwarzać obrazy normalne, trenowana wyłącznie na danych czystych. Model nie widzi żadnego ataku, więc dla niego *żaden* atak nie jest „znany” — i właśnie dlatego ma szansę wykryć atak, którego nikt mu wcześniej nie pokazał.

2 Cel eksperymentu i hipoteza

Cel. Sprawdzić, czy autoenkoder uczony wyłącznie na danych czystych wykrywa *nieznany* atak lepiej niż klasyfikator nadzorowany z Kroku 1.

Hipoteza badawcza.

- Na ataku *znanym* klasyfikatorowi (zbiór **Test_A**) klasyfikator nadzorowany będzie nieznacznie lepszy — zna dokładnie ten konkretny atak.
- Na ataku *nieznanym* (zbiór **Test_B**) autoenkoder będzie wyraźnie lepszy — nie polega na znajomości ataku, lecz na modelu normalności, który generalizuje na dowolne odstępstwo od normy.

Innymi słowy: spodziewamy się, że podejście nienadzorowane „zapłaci” niewielką cenę na ataku znanym, ale „zyska” znacznie na ataku nieznanym.

3 Metodyka

3.1 Zbiory danych

W eksperymencie wykorzystano trzy zbiory danych, opisane w tabeli 1.

Tabela 1: Zbiory danych użyte w eksperymencie.

Zbiór	Rozdzielczość	Liczność	Rola
MNIST	28×28 px	60 000 / 10 000	klasa czysta (etykieta 0)
Fashion-MNIST	28×28 px	60 000 / 10 000	źródło ataku OOD
Pythia	70×70 px	~1000 na partycję	ukryty zbiór z atakami

MNIST to zbiór odręcznie pisanych cyfr — pełni rolę danych czystych („normy”). **Fashion-MNIST** to zbiór zdjęć ubrań; służy wyłącznie jako źródło ataku typu OOD (opisanego niżej). **Pythia** to zbiór „ukryty”: udostępniony w postaci gotowych folderów z obrazami, bez ujawnienia, czym dokładnie są poszczególne ataki.

3.2 Role partycji ataków

W całym projekcie obowiązuje konwencja, w której *litera oznacza rolę ataku, a nie jego rodzaj*:

- **attack_a** — atak **znany**. Tylko klasyfikator nadzorowany może się na nim uczyć.
- **attack_b** — atak **nieznany**. Żaden model nie uczy się na nim; służy wyłącznie do testu generalizacji.

Dla zbioru **MNIST** ataki generujemy samodzielnie, więc znamy ich naturę: **attack_a** to szum gaussowski o odchyleniu standardowym $\sigma = 0,4$ dodany do każdego piksela, a **attack_b** to podmiana typu OOD — obrazy MNIST zastąpione obrazami ubrań z Fashion-MNIST. Są to ataki o zupełnie różnym charakterze, co czyni **attack_b** mocnym testem generalizacji.

Dla zbioru **Pythia** sytuacja jest inna. Partycje **clean** oraz **attack_a...attack_h** to gotowe foldery z obrazami, a *natura każdego ataku jest celowo nieujawniona*. Wiadomo jedynie tyle, ile mówi nazwa folderu: „to jakiś atak”. Na tym polega istota zbioru ukrytego — testuje on samą metodę wykrywania, a nie naszą wiedzę o konkretnym ataku.

Partycje attack_c–attack_h. Pythia udostępnia aż osiem partycji ataków, lecz treść zadania oraz Kroki 1–2 nie przypisują partycjom c–h sztywnej roli. Krok 1 korzysta wyłącznie z **attack_a** i **attack_b**. Krok 2 używa wszystkich ośmiu partycji jako *serii* kolejno dokładanych ataków w treningu progresywnym. W Kroku 3 — ponieważ autoenkoder z założenia nie trenuje na żadnym ataku — przyjęto następującą, świadomą decyzję projektową:

- **attack_a** pozostaje atakiem znanym *wyłącznie dla klasyfikatora bazowego*, aby wiernie odtworzyć układ z Kroku 1;
- **attack_b...attack_h** tworzą wspólną *baterię ataków nieznanych*, na której testujemy oba detektory.

Dzięki temu generalizację sprawdzamy nie na jednym, lecz na siedmiu różnych nieznanych atakach, co daje mocniejsze i lepiej udokumentowane wnioski. Jest to nasze rozszerzenie logiki Kroku 2, w którym wszystkie osiem partycji i tak było traktowane jako pełnoprawny materiał badawczy.

3.3 Modele

Klasyfikator bazowy (AnomalyCNN). Wierna reprodukcja modelu z Kroku 1: sieć konwolucyjna złożona z trzech bloków konwolucyjnych i dwóch warstw w pełni połączonych, zakończona pojedynczym wyjściem zwracającym prawdopodobieństwo ataku. Trenowana w sposób nadzorowany na połączeniu **clean** i **attack_a** (etykiety wykorzystywane).

Autoenkoder (ConvAutoencoder). Model o topologii „3+2” sugerowanej przez prowadzącego: enkoder składa się z trzech bloków konwolucyjnych (kanały 1→16→32→64) oraz dwóch warstw w pełni połączonych (do 128, a następnie do 32 wymiarów). Trzydziestodwuwymiarowy *kod ukryty* stanowi „wąskie gardło” sieci. Dekoder jest lustrzanym odbiciem enkodera i odtwarza obraz z kodu ukrytego. Każda warstwa konwolucyjna oraz ukryta warstwa w pełni połączona została wyposażona w normalizację wsadową (*batch normalization*) i aktywację LeakyReLU — bez tych warstw autoenkoder nie uczy się poprawnie na rzadkich, w większości czarnych obrazach MNIST. Model trenowany jest funkcją błędu średniokwadratowego (MSE) **wyłącznie na danych czystych** — nie widzi żadnego ataku ani żadnej etykiety.

Warto zaznaczyć, że enkoder zachowuje ten sam układ warstw co klasyfikator AnomalyCNN. Pozwoli to w Kroku 4 ponownie wykorzystać wytrenowany enkoder jako gotowy ekstraktor cech.

3.4 Wykrywanie anomalii przez błąd rekonstrukcji

Po treningu autoenkoder dobrze odtwarza obrazy normalne (mały błąd rekonstrukcji), lecz słabo odtwarza wszystko inne (duży błąd), ponieważ nigdy nie nauczył się odwzorowywać danych spoza normy. Miarą anomalii jest zatem *błąd rekonstrukcji*:

$$\text{Score}(x) = \|x - f(g(x))\|^2,$$

gdzie g oznacza enkoder, a f dekodery. W praktyce stosujemy średni kwadrat różnicy pikseli (jest to przeskalowanie powyższego wzoru o stałą, więc nie zmienia rankingu obrazów ani miary AUC, a jest niezależne od rozdzielczości). Im wyższy wynik, tym bardziej obraz odbiega od normy.

Wyznaczanie progu. Aby zamienić ciągły wynik na decyzję „norma / anomalia”, potrzebny jest próg. Wyznaczamy go jako 95. percentyl błędu rekonstrukcji policzonego na *czystej* części walidacyjnej. Oznacza to, że z założenia około 5% obrazów czystych przekroczy próg (fałszywe alarmy). Kluczowe jest to, że próg ustalany jest **bez oglądania jakiegokolwiek ataku** — dzięki temu metoda pozostaje w pełni nienadzorowana.

3.5 Przebieg eksperymentu

Dla każdego zbioru (MNIST oraz Pythia, niezależnie) wykonano następujące kroki:

1. Wczytanie danych czystych oraz przygotowanie `attack_a` i `attack_b`.
2. Trening klasyfikatora bazowego na `clean` $\cup$ `attack_a` (reprodukcja Kroku 1).
3. Trening autoenkodera wyłącznie na `clean` (część danych czystych odłożono jako walidację).
4. Wyznaczenie progu anomalii z walidacyjnej części danych czystych.
5. Ocena obu modeli na `Test_A = clean + attack_a` oraz `Test_B = clean + attack_b`.
6. Analiza rozszerzona: ocena obu modeli na każdym dostępnym typie ataku.

Najważniejsze parametry: rozmiar wsadu 64; klasyfikator trenowany do 15 epok (zatrzymanie po 3 epokach bez poprawy); autoenkoder do 30 epok (zatrzymanie po 5 epokach bez poprawy); wymiar kodu ukrytego 32; próg na poziomie percentyla 95; ziarno generatora liczb losowych ustawione na 42 dla powtarzalności wyników.

3.6 Metryki oceny

Skuteczność oceniano pięcioma standardowymi miarami: **Accuracy** (ogólny odsetek poprawnych decyzji), **Precision** (jaka część obrazów oznaczonych jako atak faktycznie nim była), **Recall** (jaka część prawdziwych ataków udało się wykryć), **F1** (średnia harmoniczna precyzji i czułości) oraz **AUC-ROC** (zdolność rozróżniania klas niezależna od progu).

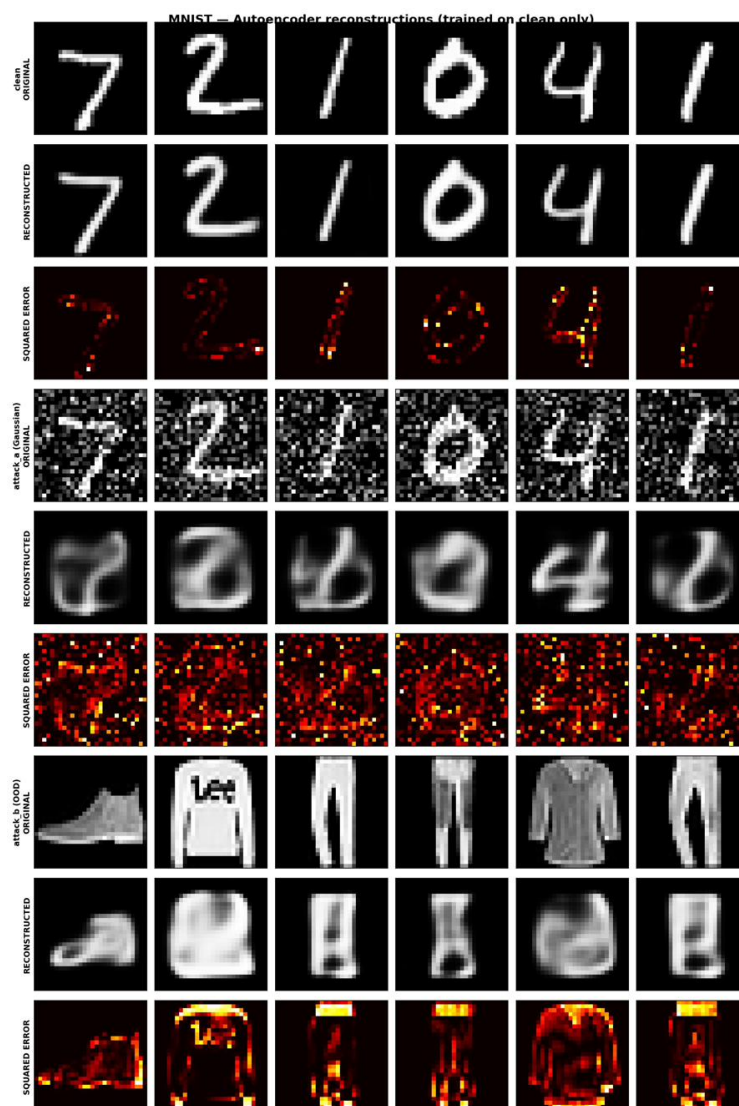
Istotne jest rozróżnienie: *AUC-ROC nie zależy od progu* — mówi, jak dobrze model *porządkuje* obrazy od najbardziej do najmniej podejrzanych, i jest najuczciwszą miarą do porównań. Z kolei *Recall i F1 zależą od progu* — pokazują, jak detektor zachowuje się w praktyce, przy konkretnej decyzji. Te dwa spojrzenia mogą się różnić, co okaże się ważne w dyskusji.

4 Wyniki

4.1 MNIST — porównanie podstawowe

Trening autoenkodera na MNIST przebiegł poprawnie: błąd rekonstrukcji spadał płynnie z 0,032 do 0,0067 (dane treningowe), osiągając najlepszą wartość walidacyjną 0,0056. Wyznaczony próg anomalii wyniósł 0,0117.

Rysunek 1 pokazuje działanie wytrenowanego autoenkodera. Obrazy czyste są odtwarzane wiernie (mapa błędu jest niemal czarna), natomiast ataki — zarówno szum gaussowski, jak i obrazy OOD — są odtwarzane słabo: sieć „na siłę” wciska je w kształt cyfry, co daje duży, dobrze widoczny błąd rekonstrukcji.



Rysunek 1: MNIST — rekonstrukcje autoenkodera (model uczony tylko na danych czystych). Wiersze: obraz wejściowy, rekonstrukcja, mapa błędu średniokwadratowego. Obrazy czyste są odtwarzane wiernie; ataki dają wyraźny błąd rekonstrukcji.

Tabela 2 zestawia oba modele na zbiorach `Test_A` i `Test_B`.

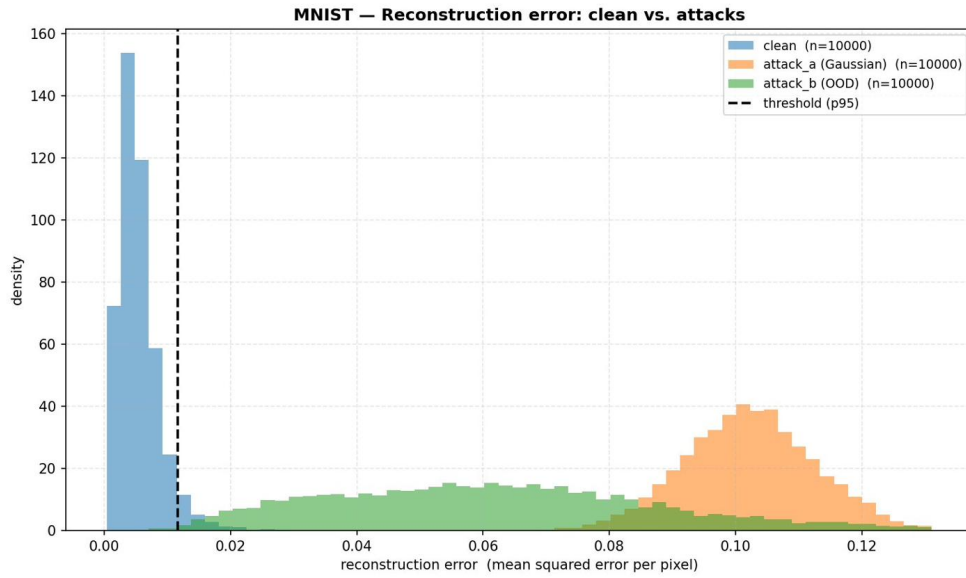
Na `Test_A` klasyfikator jest minimalnie lepszy (komplet wyników 1,000). Na `Test_B` sytuacja odwraca się drastycznie: miara F1 klasyfikatora spada do 0,244, podczas gdy autoenkoder utrzymuje 0,975. Rozkłady błędu rekonstrukcji (rysunek 2) oraz krzywe ROC (rysunek 3) ilustrują to samo zjawisko.

4.2 MNIST — analiza rozszerzona

W analizie rozszerzonej oba wytrenowane modele oceniono na sześciu typach ataków (tabela 3, rysunek 4). Klasyfikator widział w treningu tylko szum gaussowski; autoenkoder — żadnego ataku.

Tabela 2: MNIST — klasyfikator (Krok 1) kontra autoenkoder (Krok 3).

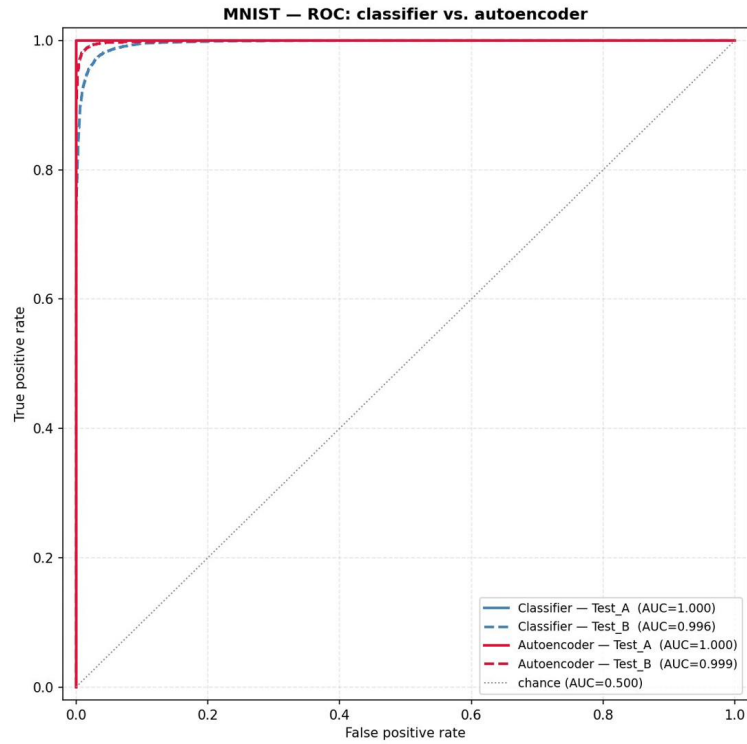
Metryka	Test _A (atak znany)		Test _B (atak nieznan)	
	Klasyfikator	Autoenkoder	Klasyfikator	Autoenkoder
Accuracy	1,000	0,976	0,570	0,974
Precision	1,000	0,954	1,000	0,954
Recall	1,000	1,000	0,139	0,997
F1-Score	1,000	0,977	0,244	0,975
AUC-ROC	1,000	1,000	0,996	0,999



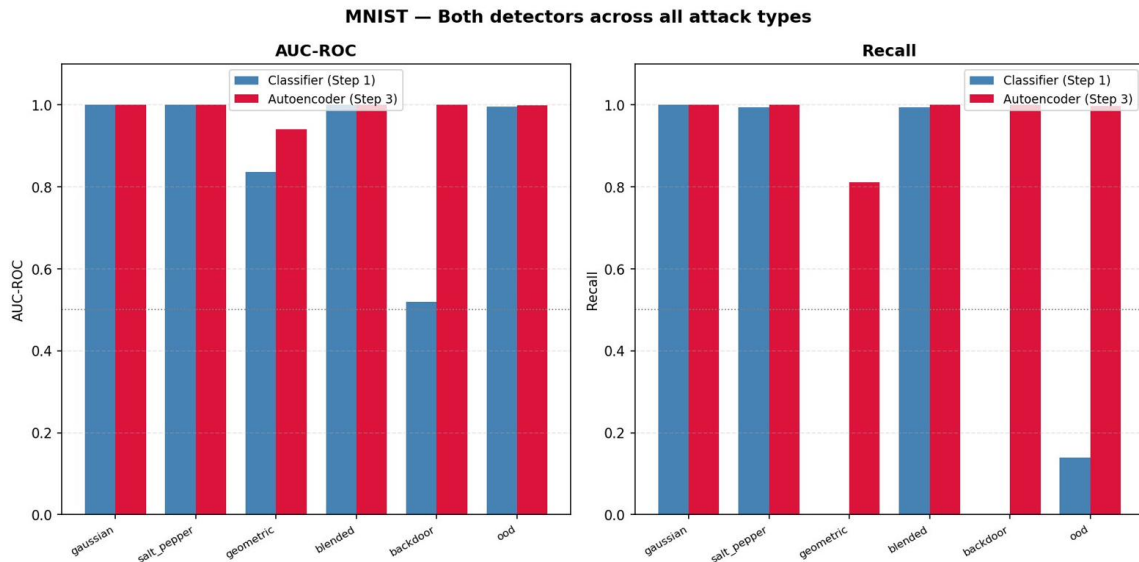
Rysunek 2: MNIST — rozkład błędu rekonstrukcji dla obrazów czystych i obu ataków. Obrazy czyste skupiają się tuż przy zerze, poniżej progu; oba ataki leżą wyraźnie po prawej stronie progu.

Tabela 3: MNIST — skuteczność obu detektorów dla każdego typu ataku.

Typ ataku	Klasyfikator (Krok 1)			Autoenkoder (Krok 3)		
	AUC	Recall	F1	AUC	Recall	F1
gaussian	1,000	1,000	1,000	1,000	1,000	0,977
salt_pepper	1,000	0,994	0,997	1,000	1,000	0,977
geometric	0,836	0,000	0,000	0,941	0,810	0,872
blended	1,000	0,994	0,997	1,000	1,000	0,977
backdoor	0,520	0,000	0,000	1,000	1,000	0,977
ood	0,996	0,139	0,244	0,999	0,997	0,975



Rysunek 3: MNIST — krzywe ROC obu detektorów na zbiorach Test_A i Test_B. Wszystkie cztery krzywe leżą wysoko, co oznacza dobrą zdolność porządkowania obrazów — różnice ujawniają się dopiero w metrykach zależnych od progu.



Rysunek 4: MNIST — porównanie obu detektorów na wszystkich typach ataków (miary AUC-ROC oraz Recall). Klasyfikator zawodzi na atakach geometric i backdoor; autoenkoder wykrywa wszystkie sześć typów.

Autoenkoder uzyskuje identyczne wartości F1 (0,977) dla ataków gaussian, salt_pepper, blended i backdoor. Nie jest to błąd — wynika to z faktu, że dla tych czterech ataków czułość (Recall) osiąga 1,000, więc o pozostałych metrykach decyduje już tylko stały odsetek fałszywych alarmów na tym samym zbiorze obrazów czystych. Jedynie atak geometric wyróżnia się niższym wynikiem (F1 0,872), ponieważ część zniekształconych cyfr wciąż przypomina poprawne cyfry i zostaje odtworzona na tyle dobrze, że nie przekracza progu.

4.3 Pythia — wyniki

Trening autoenkodera na zbiorze Pythia został zatrzymany przez mechanizm wczesnego stopu po 17 epokach, na błędzie rekonstrukcji rzędu 0,077 — po krótkim początkowym spadku strata szybko przestała maleć. Tabela 4 przedstawia wyniki podstawowe, a tabela 5 — miarę AUC dla wszystkich ośmiu partycji.

Tabela 4: Pythia — klasyfikator kontra autoenkoder.

Metryka	Test_A		Test_B	
	Klasyfikator	Autoenkoder	Klasyfikator	Autoenkoder
Accuracy	0,500	0,523	0,500	0,513
Precision	0,000	0,622	0,000	0,576
Recall	0,000	0,115	0,000	0,095
F1-Score	0,000	0,194	0,000	0,163
AUC-ROC	0,570	0,528	0,571	0,520

Tabela 5: Pythia — AUC-ROC obu detektorów dla wszystkich partycji ataków.

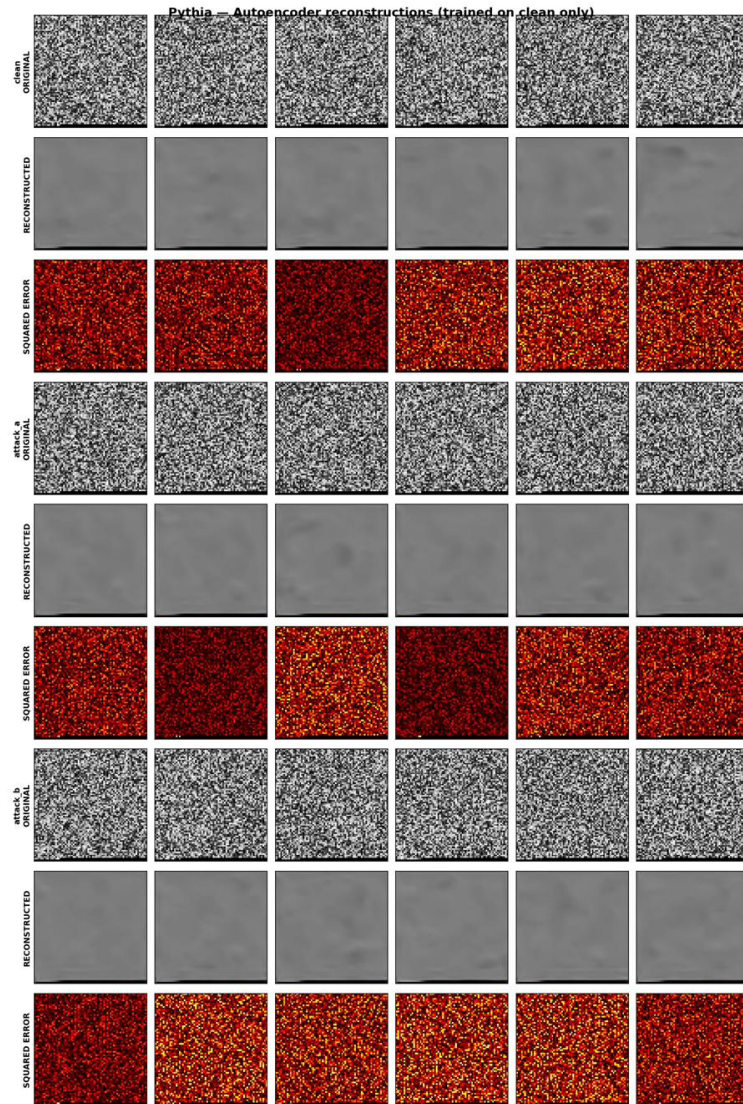
Partycja	a	b	c	d	e	f	g	h
Klasyfikator	0,570	0,571	0,589	0,521	0,562	0,527	0,526	0,560
Autoenkoder	0,528	0,520	0,546	0,503	0,519	0,502	0,512	0,535

Wszystkie wartości AUC mieszczą się w wąskim przedziale 0,50–0,59, czyli na poziomie losowego zgadywania. Rysunek 5 wyjaśnia, dlaczego tak się dzieje, a rysunek 6 pokazuje całkowity brak separacji między obrazami czystymi a atakami.

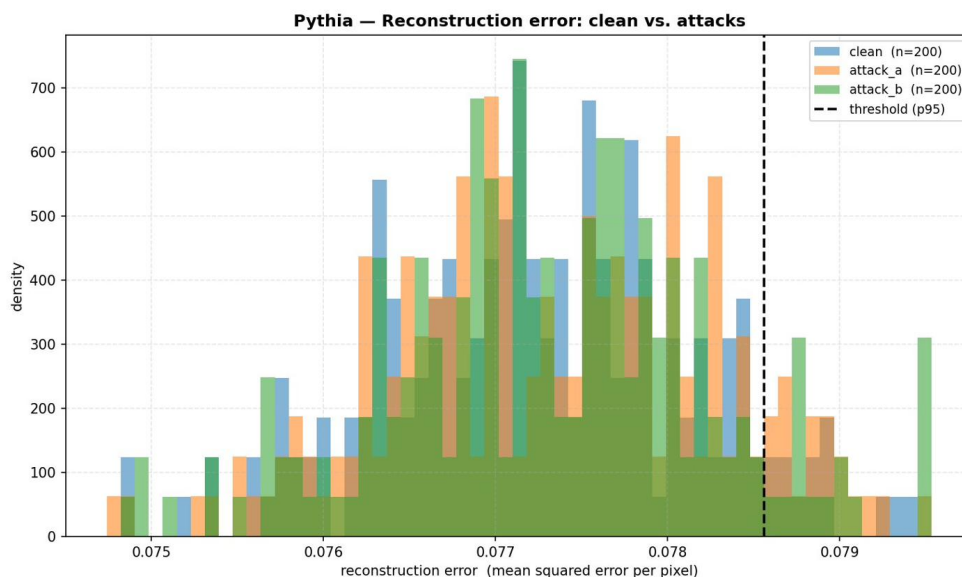
5 Dyskusja

5.1 Atak znany — cena podejścia nienadzorowanego

Na zbiorze Test_A klasyfikator osiągnął komplet wyników 1,000, a autoenkoder nieznacznie mniej (Accuracy 0,976, F1 0,977). Ta różnica jest zgodna z oczekiwaniami i ma prostą przyczynę. Klasyfikator widział szum gaussowski w treningu, więc rozpoznaje go bezbłędnie. Autoenkoder nie widział go nigdy — wykrywa go pośrednio, jako odstępstwo od normy. Niewielki ubytek skuteczności bierze się niemal w całości z przyjętego progu na poziomie percentyla 95: z definicji oznacza on około 5% fałszywych alarmów na danych czystych, co obniża dokładność. Jest to świadomie zaakceptowana *cena podejścia nienadzorowanego* — i, jak pokazuje kolejna sekcja, cena bardzo niska wobec uzyskanej korzyści.



Rysunek 5: Pythia — rekonstrukcje autoenkodera. Obrazy (czyste i ataki) wyglądają jak czysty szum. Autoenkoder zwraca dla każdego wejścia niemal identyczne, jednolicie szare rozmycie, a mapa błędów jest taka sama dla danych czystych i ataków.



Rysunek 6: Pythia — rozkład błędu rekonstrukcji. Rozkłady dla danych czystych oraz obu ataków całkowicie się pokrywają — nie istnieje próg, który rozdzieliłby te klasy.

5.2 Atak nieznany — potwierdzenie hipotezy

Na zbiorze **Test_B** (atak OOD, nieznany obu modelom) różnica jest ogromna. Autoenkoder uzyskał F1 0,975, klasyfikator zaledwie 0,244 — a jego czułość spadła do 0,139, co oznacza, że przeoczył blisko 86% ataków.

Na pierwszy rzut oka zaskakuje wysokie AUC klasyfikatora na **Test_B** (0,996). AUC mierzy jednak tylko zdolność *porządkowania* obrazów — i tę klasyfikator zachowuje, ustawia obrazy OOD wyżej niż czyste cyfry. Zawodzi próg decyzyjny: klasyfikator nauczył się reagować na *szum*, a obrazy OOD nie są zaszumione — są po prostu inne. W efekcie ich prawdopodobieństwo, choć wyższe niż dla czystych cyfr, w większości nie przekracza progu 0,5 i obrazy te zostają uznane za normalne.

Stąd kluczowy wniosek: o praktycznej skuteczności detektora decyduje jego działanie przy konkretnym progu — i właśnie tam klasyfikator się załamuje, ponieważ jego skala wyników nie przenosi się na nowy typ ataku. Autoenkoder nie ma tego problemu: jego próg pochodzi z danych czystych, a błąd rekonstrukcji ataku jest na tyle duży, że naturalnie go przekracza (por. rysunek 2). Hipoteza badawcza zostaje zatem **potwierdzona**.

5.3 Co wykrywa każda z metod

Analiza rozszerzona (tabela 3) pokazuje wyraźny podział.

Klasyfikator radzi sobie znakomicie z atakami „szumowymi” — gaussian, salt_pepper i blended — ponieważ wszystkie zaburzają wartości pikseli podobnie do szumu, który widział w treningu. Zawodzi natomiast tam, gdzie atak ma inny charakter:

- *geometric* — zniekształcenie geometryczne nie dodaje żadnego szumu, jedynie wygina cyfrę; klasyfikator nie ma jak go rozpoznać (Recall 0,000).
- *backdoor* — mały biały kwadrat w rogu; klasyfikator w ogóle go nie zauważa (AUC 0,520, czyli poziom losowy).

Autoenkoder wykrywa wszystkie sześć typów ataków (AUC od 0,941 wzwyż). Szczególnie pouczający jest przypadek *backdoor*. Autoenkoder uczył się wyłącznie czystych cyfr, w których róg obrazu jest zawsze czarny — nigdy nie widział tam białego kwadratu i nie potrafi go

odtworzyć. Cały błąd rekonstrukcji skupia się więc na obszarze triggera i pewnie przekracza próg. Atak, który całkowicie pokonał klasyfikatora, jest dla autoenkodera łatwy. Nieco trudniejszy okazał się atak *geometric* (F1 0,872): wygięta cyfra to wciąż „cyfropodobny” kształt, więc autoenkoder potrafi częściowo ją odtworzyć i część takich obrazów nie przekracza progu. Mimo to autoenkoder i tak wypada tu zdecydowanie lepiej niż klasyfikator.

5.4 Pythia — dlaczego obie metody zawodzą

Na zbiorze Pythia ani klasyfikator, ani autoenkoder nie osiągnęły wyników lepszych niż losowe (AUC w przedziale 0,50–0,59 dla wszystkich ośmiu partycji). Przyczynę dobrze widać na rysunku 5: obrazy Pythii — zarówno czyste, jak i zaatakowane — wyglądają jak czysty szum, pozbawiony widocznej struktury. Autoenkoder nie ma czego skompresować, więc dla każdego wejścia zwraca to samo jednolicie szare rozmycie, a błąd rekonstrukcji jest niemal stały i identyczny dla danych czystych oraz ataków. W konsekwencji rozkłady błędu całkowicie się pokrywają (rysunek 6) i żaden próg nie jest w stanie rozdzielić klas.

Wynik ten jest negatywny, ale wartościowy i spójny z wnioskami Kroku 2, który również wykazał nieskuteczność klasyfikatora na zbiorze Pythia. Co istotne, Krok 2 wskazywał metody nienadzorowane (w tym autoenkoder) jako możliwe rozwiązanie problemu Pythii. Krok 3 ten kierunek sprawdził i pokazał, że samo przejście na autoenkoder *nie wystarcza*: jeśli różnica między danymi czystymi a atakiem nie jest zakodowana w statystyce pikseli, model operujący na surowych pikselach jej nie wykryje — niezależnie od tego, czy jest nadzorowany, czy nie.

6 Wnioski

1. **Hipoteza została potwierdzona dla zbioru MNIST.** Autoenkoder uczony wyłącznie na danych czystych wykrywa nieznany atak znacznie lepiej niż klasyfikator nadzorowany (F1 0,975 wobec 0,244 na `Test_B`).
2. **Cena podejścia nienadzorowanego jest niska.** Na ataku znanym autoenkoder ustępuje klasyfikatorowi tylko nieznacznie (F1 0,977 wobec 1,000), a różnica wynika głównie z przyjętego progu fałszywych alarmów.
3. **Dwa podejścia, dwie filozofie.** Klasyfikator uczy się konkretnego ataku i działa świetnie tylko w jego pobliżu. Autoenkoder uczy się normy i traktuje każde od niej odstępstwo jako anomalię — dlatego generalizuje na ataki nowego typu, w tym na atak typu backdoor, który całkowicie pokonał klasyfikatora.
4. **AUC to nie wszystko.** Wysokie AUC klasyfikatora na ataku nieznanym pokazało, że model „widzi” różnicę, lecz jego próg decyzyjny się nie przenosi. O praktycznej przydatności detektora decydują metryki zależne od progu — i tu przewaga autoenkodera jest jednoznaczna.
5. **Zbiór Pythia pozostaje poza zasięgiem metod pikselowych.** Obie metody działają na poziomie losowym, ponieważ obrazy Pythii są szumopodobne i nieściśliwe. Rozwiązanie wymagałoby cech innych niż surowe piksele (np. analizy w dziedzinie częstotliwości).
6. **Przygotowanie do Kroku 4.** Enkoder autoenkodera ma ten sam układ warstw co klasyfikator AnomalyCNN, dzięki czemu może zostać ponownie wykorzystany jako ekstraktor cech w systemie hybrydowym budowanym w kolejnym kroku.

Załącznik — pliki wynikowe

- `step3.py` — skrypt przeprowadzający eksperyment.

- `lib.py` — współdzielone narzędzia: model `ConvAutoencoder`, funkcje treningu i oceny autoenkodera.
- `faza3_wyniki_autoenkodera.json` — pełne wyniki liczbowe.
- `plots/step3/` — komplet wykresów: rekonstrukcje, rozkłady błędu rekonstrukcji, krzywe ROC, porównania metryk oraz analiza dla poszczególnych typów ataków, osobno dla MNIST i Pythii.